

BEAM: Encoding-Aware Greedy Merging for Scalable Flat Graph Summarization

Boyang Zhang
Hunan University
Changsha, Hunan, China
zhangboyang@hnu.edu.cn

Guoqing Xiao
Hunan University
Changsha, Hunan, China
xiaoguoqing@hnu.edu.cn

Yuedan Chen
Central South University
Changsha, Hunan, China
chenyuedan@hnu.edu.cn

Zhuo Tang
Hunan University
Changsha, Hunan, China
ztang@hnu.edu.cn

Xiaofei Zhang
University of Memphis
Memphis, Tennessee, USA
Xiaofei.Zhang@memphis.edu

Kenli Li
Hunan University
Changsha, Hunan, China
lkl@hnu.edu.cn

ABSTRACT

Flat correction-set graph summarization represents a graph with supernodes, superedges, and correction edges, enabling exact reconstruction with a compact flat summary. Scalable methods in this model follow an iterative divide-merge-encode pipeline, and the repeated merge stage is the main scalability challenge because candidate pairs must be generated, evaluated under the encoding objective, selected without conflicts, and applied while the summary state evolves.

We present BEAM, an encoding-aware greedy merging algorithm for scalable flat graph summarization. BEAM makes flat summarization faster by decoupling candidate discovery from objective-faithful merge scoring: lightweight profiles only prune the search space, while retained candidates are scored by exact local correction-set gain. Concretely, BEAM builds a sparse candidate graph within each divide group, applies greedy positive-gain matching within groups, and defers partition updates to the end of each group block. Retained gain evaluations are packed into batched scoring tasks, with CUDA used as one backend for that scoring step.

On ten common datasets, BEAM is faster than SWeG on all datasets and faster than SLUGGER on nine of them, with geometric mean speedups of $2.69\times$ over SWeG and $2.29\times$ over SLUGGER. BEAM also produces smaller relative output size than SWeG on all ten common datasets, while compression results against the hierarchical SLUGGER baseline remain mixed. Runtime breakdowns of the batch-ea-blocked CUDA implementation show that merge-related work accounts for 50.0%–97.9% of BEAM’s execution time, confirming that BEAM targets a major, and often dominant, cost in the flat summarization pipeline.

PVLDB Reference Format:

Boyang Zhang, Guoqing Xiao, Yuedan Chen, Zhuo Tang, Xiaofei Zhang, and Kenli Li. BEAM. PVLDB, TBD(TBD): TBD, Under review.

doi:TBD

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/githubofzby/beam>.

1 INTRODUCTION

Large graphs are now routinely used to represent web pages, social interactions, communication records, biological relations, knowledge bases, and many other forms of relational data. Directly storing and processing such graphs is expensive because graph analytics repeatedly access large, sparse, and irregular adjacency structures. A broad body of graph compression work therefore seeks to reduce storage cost through vertex ordering, reference encoding, interval encoding, locality-aware layout, recursive partitioning, and compressed graph indexes [1, 3, 5, 8, 10]. These techniques primarily encode a fixed input graph compactly. Graph summarization takes a complementary approach: it constructs a smaller structural representation that groups vertices and records the information needed to reconstruct or approximate the original graph [18].

This paper focuses on flat correction-set graph summarization, where a graph is represented by disjoint supernodes, superedges, and positive and negative correction edges [20, 22]. This model provides explicit reconstruction semantics and can serve as a compact preprocessing representation for large graphs. Scalable methods in this model commonly follow an iterative divide-merge-encode pipeline: divide restricts the search space by grouping current supernodes, merge selects beneficial supernode pairs, and encode materializes the final superedges and correction sets.

The repeated merge stage presents both a computational and an algorithmic challenge. It is invoked over many iterations while the summary state is evolving. Each accepted merge changes supernode memberships, quotient neighborhoods, pair capacities, future candidate pairs, and the encoding costs of subsequent decisions. The algorithm therefore does not optimize one static candidate graph, but operates over a sequence of state-dependent candidate graphs whose

Proceedings of the VLDB Endowment, Vol. TBD, No. TBD ISSN 2150-8097.
doi:TBD

vertices, edges, and weights are repeatedly regenerated. Exhaustively evaluating all possible pairs is infeasible; however, allowing a lightweight structural proxy to determine merge acceptance can detach the search trajectory from the correction-set objective. The central problem is thus to sparsify this evolving search space aggressively while preserving objective-faithful decisions for the candidates that remain.

We present BEAM, an algorithm–system co-design for scalable flat graph summarization built around one principle: *approximate the search space, but not the merge objective*. BEAM uses lightweight quotient profiles to construct a bounded sparse candidate graph, evaluates each retained pair under the exact local correction-set objective, and exposes the exact-admissible candidate edges to descending-gain conflict resolution. Selected endpoint-disjoint merges are committed under a frozen block state, after candidate construction, exact evaluation, and matching for the block have completed. BEAM thereby combines profile-based candidate sparsification, encoding-aware exact gain evaluation, conflict-free matching, and blocked merge execution without changing the underlying flat correction-set representation.

This design separates three responsibilities. The *proposal layer* decides where a limited exact-evaluation budget is spent by grouping current supernodes and retaining a sparse set of profile-induced pairs. The *decision layer* assigns retained edges their exact correction-set gains and selects a high-value conflict-free subset. The *execution layer* maintains the evolving summary state and realizes the same proposal and decision semantics efficiently. This separation turns the retained candidate graph into a clean algorithm–system interface: its topology is determined by sparse proposal, its weights are defined by the correction-set objective, and its selected edges are applied only at an explicit block boundary.

The resulting formulation provides a precise scope for both exactness and approximation. For every retained candidate, the locally computed gain equals the corresponding change in global partition cost under the current state. Once exact evaluation has fixed a retained weighted graph, descending-gain greedy matching is a 1/2-approximation to maximum-weight matching on that same graph. This guarantee concerns conflict resolution on a fixed retained graph; it does not extend to pairs omitted by sparsification or to the globally optimal final summary. Likewise, endpoint-disjointness gives a well-defined blocked partition update, but does not require the isolated gains of different selected merges to be additive.

The same separation guides the implementation. A unified exact cost oracle connects local merge evaluation, global partition cost, and final correction-set accounting. A persistent quotient representation maintains current supernode statistics and avoids repeatedly rebuilding their neighborhoods from the original graph. Retained-pair evaluation is organized as a batched, backend-neutral scoring interface that admits multicore CPU execution and an optional CUDA realization. These components provide a semantics-preserving execution substrate for BEAM rather than alternative definitions of the algorithm.

BEAM improves the merge process without changing the expressive power of the output representation. It remains within the same flat correction-set model as scalable flat methods such as SWeG [22]. In contrast, SLUGGER introduces a hierarchical representation with nested supernodes and positive and negative edges between hierarchical supernodes [15]. We therefore distinguish same-model comparisons, which evaluate different construction strategies for flat correction-set summaries, from cross-model comparisons that provide context across representations with different expressive power.

These observations lead to the central research question of this work: *Can a flat graph summarizer sparsify a sequence of evolving merge-search spaces while retaining exact encoding-aware decisions, effective conflict resolution, and well-defined blocked execution?* BEAM answers this question by assigning candidate discovery, merge decision, and state mutation to distinct but composable layers.

This paper makes the following contributions.

- We propose BEAM, an objective-faithful sparse merging algorithm for flat correction-set graph summarization. BEAM organizes summarization as optimization over a sequence of dynamically regenerated candidate graphs, combining profile-based candidate sparsification, exact encoding-aware edge weights, conflict-free selection, and blocked partition updates.
- We formalize the decision problem induced by each retained candidate graph. We show that each retained edge weight is an exact local rewriting of the corresponding global partition-cost change. For a fixed retained exact-admissible graph, BEAM’s descending-gain matching is a 1/2-approximation to maximum-weight matching on that graph, while the blocked semantics explicitly separate isolated edge weights from realized batch effects.
- We develop a semantics-preserving execution substrate for the same BEAM algorithm. A unified exact cost oracle, persistent quotient state, and batched scoring interface reduce repeated state reconstruction and expose exact retained-edge evaluation to multicore CPU and optional CUDA backends without redefining candidate construction, selection, or block-commit semantics.
- We evaluate BEAM through complementary evidence: exact-objective and lossless-output validation, candidate-sparsification and exact-validation behavior, deterministic work eliminated by persistent state, controlled compression–runtime tradeoffs, and negative ablations that expose the limits of optimizing candidate counts or isolated local decisions. CUDA is evaluated separately as an optional scoring backend.

2 PROBLEM FORMULATION AND DESIGN PRINCIPLES

2.1 Flat Correction-Set Graph Summarization

Let the input be a simple undirected graph $G = (V, E)$. A flat graph summary replaces the original vertices with a partition $\mathcal{S} = \{S_1, \dots, S_r\}$, where each $S_i \subseteq V$, $S_i \cap S_j = \emptyset$ for $i \neq j$, and $\bigcup_i S_i = V$. Each set S_i is called a supernode. The partition is flat because every original vertex belongs to exactly one supernode and the summary does not maintain nested or overlapping supernodes. This representation is the basis of correction-set graph summarization and is also used by scalable flat methods such as SWeG [20, 22].

Given a flat partition, the summary stores a set of superedges P together with two correction sets C^+ and C^- . A superedge between two supernodes represents the decision to encode the corresponding vertex-pair region densely at the summary level. Positive corrections C^+ store original edges that are not implied by the selected superedges, while negative corrections C^- store missing edges that would otherwise be implied by those superedges. We write the resulting summary as

$$\Sigma = (\mathcal{S}, P, C^+, C^-).$$

The representation has explicit reconstruction semantics: the original graph is recovered by expanding the superedges and then applying the positive and negative corrections.

For two current supernodes X and Y , let $e(X, Y)$ be the number of original graph edges represented by the pair. The number of possible undirected edges covered by the pair is

$$\text{cap}(X, Y) = \begin{cases} |X||Y|, & X \neq Y, \\ |X|(|X| - 1)/2, & X = Y. \end{cases} \quad (1)$$

The encoder has two local choices. It can represent the pair sparsely by storing the existing edges directly:

$$L_{\text{sparse}}(X, Y) = e(X, Y). \quad (2)$$

Alternatively, it can represent the pair densely by storing one superedge and recording the missing edges as negative corrections:

$$L_{\text{dense}}(X, Y) = 1 + \text{cap}(X, Y) - e(X, Y), \quad (3)$$

where the constant term accounts for the superedge itself. The local correction-set cost is therefore

$$L(X, Y) = \min \{L_{\text{sparse}}(X, Y), L_{\text{dense}}(X, Y)\}. \quad (4)$$

For a fixed partition $\mathcal{S}$, we define its exact correction-set cost as the sum over all unordered supernode pairs, including self-pairs:

$$\text{Cost}(\mathcal{S}) = \sum_{X \in \mathcal{S}} L(X, X) + \sum_{\{X, Y\} \in \binom{\mathcal{S}}{2}} L(X, Y). \quad (5)$$

Here, $\binom{\mathcal{S}}{2}$ denotes the unordered pairs of distinct supernodes. Equation (5) is the minimum correction-set cost of the fixed partition under the pairwise sparse-or-dense encoding rule in Equation (4). It is also the objective according to which BEAM evaluates candidate merges.

Table 1: Frequently used notation.

Symbol	Meaning
$G = (V, E)$	Input graph.
$\mathcal{S}$	Current flat partition into supernodes.
Σ	Flat summary $(\mathcal{S}, P, C^+, C^-)$.
X, Y, Z	Current supernodes.
P	Superedge set in the final flat summary.
C^+, C^-	Positive and negative correction sets.
W_X	Sparse neighborhood profile of supernode X .
Q	Divide group of active supernodes.
C_Q	Retained candidate-pair set inside group Q .
C_Q^+	Positive-gain edges retained from C_Q .
M_{XY}	Merged supernode $X \cup Y$.
$\mathcal{S} \downarrow (X, Y)$	Partition after merging X and Y .
$\mathcal{M}_Q$	Conflict-free merge set selected inside group Q .
$\text{cap}(X, Y)$	Number of possible original edges represented by pair (X, Y) .
$L(X, Y)$	Local correction-set cost for pair (X, Y) .
$\text{Cost}(\mathcal{S})$	Exact correction-set cost of partition $\mathcal{S}$.
$\text{gain}(X, Y)$	Exact encoding gain of merging X and Y .

BEAM changes neither this objective nor the expressive power of the representation. Its output remains a flat partition with superedges and correction edges rather than a hierarchical summary. Its algorithmic task is to search efficiently over partitions induced by successive supernode merges.

Table 1 summarizes the notation used throughout the paper.

2.2 Evolving Divide-and-Merge Construction

Flat correction-set summaries are commonly constructed through an iterative divide-and-merge pipeline. Starting from an initial partition, the algorithm repeatedly groups currently active supernodes, searches for promising pairs inside each group, evaluates whether merging a pair reduces the encoding cost, and applies accepted merges to update the partition.

SWeG follows this general strategy and explicitly separates dividing, merging, and encoding: shingle-based grouping restricts the candidate search space, the merging step greedily considers within-group pairs, and the final encoding step materializes superedges and corrections [22]. In such a pipeline, a cheap structural proxy is useful because evaluating the true correction-set gain for every possible pair is too expensive. SWeG therefore uses SuperJaccard to choose a candidate partner inside each group and then applies a saving threshold to decide whether to accept the merge [22].

Later scalable summarizers follow the same broad divide-and-merge idea but alter either the candidate-search mechanism or the output representation. LDME-style methods and related scalable summarizers use locality-sensitive hashing to find promising merges efficiently [21, 28]. SLUGGER also reduces candidate search, but changes the underlying

representation to a hierarchical model and performs greedy merges under that model [15]. BEAM instead keeps the flat correction-set representation fixed and studies how candidate search, objective evaluation, conflict resolution, and partition updates should be organized within that model.

The difficulty is not merely that a single candidate set may be large. The search problem evolves after every accepted block of merges. A merge changes supernode memberships, quotient neighborhoods, pair capacities, future candidate edges, and the gains assigned to those edges. The algorithm therefore operates over a sequence of dynamically regenerated candidate graphs rather than one static weighted graph. Exhaustive exact evaluation of every possible pair is infeasible, but allowing a lightweight search proxy to determine merge acceptance can detach the resulting trajectory from the actual correction-set objective.

A key limitation of similarity-based or proxy-based search is that the proxy is not itself the encoding objective. A profile score measures whether two supernodes have related neighborhood patterns, whereas the actual compression effect depends on supernode sizes, local pair capacities, edge counts, affected neighboring supernodes, and changes between sparse and dense encoding choices. A highly ranked pair may therefore produce no encoding reduction, while a moderately ranked pair may yield positive gain because savings accumulate over several affected pair costs.

As a simple running example, first consider two singleton supernodes A and B . Suppose each has one edge to Z_1 and one edge to Z_2 , where Z_1 and Z_2 each contain at least two vertices, and assume there are no internal edges and no edge between A and B . A Jaccard-style profile similarity would rank A and B highly because their external profiles are identical. However, the external cost before merging is

$$L(A, Z_1) + L(B, Z_1) + L(A, Z_2) + L(B, Z_2) = 1 + 1 + 1 + 1 = 4.$$

After merging A and B , each merged pair is still encoded sparsely:

$$L(A \cup B, Z_1) = 2, \quad L(A \cup B, Z_2) = 2.$$

The external cost remains 4, and the endpoint and self-pair terms do not change under the assumptions above. Thus, this highly ranked pair has zero encoding gain.

Conversely, consider two supernodes C and D , each of size 3, and four neighboring supernodes $Z_1, \dots, Z_4$, each also of size 3. Suppose C has five edges to each of Z_1, Z_2, Z_3 , while D has five edges to each of Z_1, Z_2, Z_4 , again with no internal edges and no edge between C and D . Their profiles overlap only on Z_1 and Z_2 , so their profile similarity is moderate. Yet for each shared neighbor Z_i , the pre-merge cost is

$$L(C, Z_i) + L(D, Z_i) = 5 + 5 = 10,$$

whereas after merging C and D , the dense encoding cost becomes

$$L(C \cup D, Z_i) = 1 + 18 - 10 = 9.$$

The merge saves one unit on each shared neighbor, and these savings accumulate over the affected neighborhood.

This example motivates the central design principle of BEAM: lightweight profiles determine which pairs receive exact evaluation, but they do not certify or accept merges. Every retained candidate is validated using the correction-set objective defined above.

2.3 Exact Local-to-Global Encoding Gain

Consider a candidate merge (X, Y) under the current partition $\mathcal{S}$. Let

$$M_{XY} = X \cup Y$$

be the merged supernode, and denote the resulting partition by

$$\mathcal{S} \downarrow (X, Y) = (\mathcal{S} \setminus \{X, Y\}) \cup \{M_{XY}\}. \quad (6)$$

Only encoding terms incident to X or Y can change under this partition transformation. All other supernode-pair costs are identical in $\mathcal{S}$ and $\mathcal{S} \downarrow (X, Y)$ and cancel when the cost difference is computed. We define the affected scope as

$$\Omega(X, Y) = \{X, Y\} \cup \{Z \in \mathcal{S} : e(X, Z) > 0 \text{ or } e(Y, Z) > 0\}. \quad (7)$$

The external affected neighborhood excludes the two merge endpoints:

$$N_{XY} = \Omega(X, Y) \setminus \{X, Y\}. \quad (8)$$

Before merging X and Y , the affected terms contribute

$$L_{\text{before}}(X, Y) = L(X, X) + L(Y, Y) + L(X, Y) + \sum_{Z \in N_{XY}} (L(X, Z) + L(Y, Z)). \quad (9)$$

After replacing X and Y by M_{XY} , the corresponding terms contribute

$$L_{\text{after}}(X, Y) = L(M_{XY}, M_{XY}) + \sum_{Z \in N_{XY}} L(M_{XY}, Z). \quad (10)$$

The exact local encoding gain is

$$\text{gain}(X, Y) = L_{\text{before}}(X, Y) - L_{\text{after}}(X, Y). \quad (11)$$

The locality of Equation (11) is not an approximation. It gives exactly the change in the global partition objective:

$$\text{gain}(X, Y) = \text{Cost}(\mathcal{S}) - \text{Cost}(\mathcal{S} \downarrow (X, Y)). \quad (12)$$

To see this, partition the terms in Equation (5) into those incident to X or Y and all remaining terms. The latter terms are unchanged by the merge. The former terms are exactly $L_{\text{before}}(X, Y)$ before the merge and $L_{\text{after}}(X, Y)$ afterward.

A positive value of $\text{gain}(X, Y)$ therefore means that applying this merge in isolation strictly decreases the current flat correction-set cost. BEAM uses this exact value as the weight of a retained candidate edge and excludes non-positive candidates before matching.

Equation (12) concerns one merge evaluated against one partition state. When several endpoint-disjoint merges are scored against a frozen state and later committed together, their isolated gains need not sum exactly to the realized batch-level cost change. In particular, pair costs between the endpoints of two different selected merges may change jointly after both merges are applied. BEAM therefore treats exact local gain as an objective-faithful edge weight for block-local

selection, not as a certificate of global optimality for the final summary.

2.4 Retained Candidate Graph and Merge Selection

Consider one divide group Q . Candidate discovery produces a deduplicated retained set

$$C_Q \subseteq \binom{Q}{2}.$$

Each retained pair $\{X, Y\} \in C_Q$ is evaluated under the current partition using Equation (11). The positive-gain edge set is

$$C_Q^+ = \{\{X, Y\} \in C_Q : \text{gain}(X, Y) > 0\}. \quad (13)$$

These edges induce a weighted candidate graph

$$H_Q = (Q, C_Q^+, w), \quad w(X, Y) = \text{gain}(X, Y).$$

Given this fixed retained graph, the block-local merge-selection subproblem is

$$\begin{aligned} \max_{\mathcal{M} \subseteq C_Q^+} \quad & \sum_{\{X, Y\} \in \mathcal{M}} w(X, Y) \\ \text{s.t.} \quad & \text{each } X \in Q \text{ is incident to at most one edge in } \mathcal{M}. \end{aligned} \quad (14)$$

Thus, conflict resolution on a fixed retained graph is a maximum-weight matching problem. Exact maximum-weight matching is polynomial-time solvable [11]. In BEAM, however, Equation (14) is only a repeatedly generated local subproblem. After selected merges are applied, the supernodes, candidate edges, and edge weights change, and candidate generation and exact evaluation must be performed again.

BEAM therefore processes the edges of C_Q^+ in descending gain order and greedily accepts an edge when neither endpoint has already been used. This produces a deterministic conflict-free matching while avoiding the cost of repeatedly solving exact weighted matching on many small, dynamically regenerated candidate graphs. On a fixed retained positive-gain graph, the standard charging argument gives a $1/2$ -approximation to maximum-weight matching on that same graph. The guarantee is deliberately restricted to the conflict-resolution step after candidate sparsification and exact gain evaluation; it is not an approximation guarantee for the globally optimal flat summary.

2.5 Algorithmic Separation and Execution Boundary

The preceding formulation separates three responsibilities in the repeated merge process.

First, the *proposal layer* controls the search budget. It groups the current supernodes and uses lightweight structural profiles to construct the retained candidate sets C_Q . Proposal quality affects which portions of the evolving merge space are explored, but a proxy score is not used as a correctness certificate.

Second, the *decision layer* assigns every retained edge its exact correction-set gain, removes non-positive edges, and

constructs a conflict-free matching on the resulting weighted candidate graph. This layer determines which proposed merges are accepted.

Third, the *execution layer* applies the selected merges and maintains the partition state from which the next sequence of candidate graphs will be generated. BEAM uses blocked execution: the candidates in a block are prepared, scored, and matched against a frozen partition state, and the selected endpoint-disjoint merges are committed after block-local selection has completed. This boundary keeps candidate discovery, objective evaluation, conflict resolution, and state mutation conceptually separate.

The separation also exposes backend-neutral parallelism. Once a retained pair and the required local statistics have been prepared, its exact gain can be evaluated independently of other retained pairs against the same frozen partition state. Retained-pair evaluations can therefore be packed into batched scoring units and processed by multicore CPU workers or by an optional CUDA backend. The surrounding candidate construction, conflict resolution, and evolving-state updates retain irregular control and mutation dependencies of the kind commonly encountered in parallel graph workloads [25].

Backend selection changes only how the exact weights of retained candidate edges are evaluated. It does not change the candidate set, the correction-set objective, positive-gain filtering, matching policy, or blocked merge semantics. CUDA is consequently an optional scoring backend within BEAM rather than part of the definition of the BEAM algorithm.

3 BEAM ALGORITHM

3.1 Overview

BEAM repeatedly constructs and optimizes sparse candidate graphs over the evolving flat partition. In iteration t , it regenerates disjoint divide groups from the active supernodes, builds a bounded retained candidate set inside each group, assigns retained edges their exact objective gains, and greedily selects an endpoint-disjoint subset from the exact-admissible graph defined in Section 2.4. Groups are processed in blocks: all proposal and decision steps within a block use one frozen partition state, and the selected merges are committed together at the block boundary.

Algorithm 1 gives the complete procedure. The candidate budget k bounds per-source retention, while τ_t specifies the exact-admissibility threshold for iteration t . The data structure used to maintain the quotient state and the backend used to score retained edges are deferred to Section 4.

3.2 Profile-Based Candidate Sparsification

Candidate sparsification determines which pairs receive exact evaluation. BEAM first restricts eligibility through adaptive divide groups and then retains at most k proxy-ranked partners per source supernode. The resulting deduplicated set C_Q is the sparse edge set presented to the exact decision stage.

Algorithm 1: BEAM

Input: graph $G = (V, E)$; iteration budget T ; candidate budget k ; thresholds $\{\tau_t\}_{t=1}^T$.
Output: flat correction-set summary $\Sigma = (S, P, C^+, C^-)$.

- 1: Initialize $S \leftarrow \{\{v\} : v \in V\}$ and $A \leftarrow S$.
- 2: **for** $t = 1$ to T **do**
- 3: $Q_t \leftarrow \text{ADAPTIVEMULTIHASHDIVIDE}(A)$.
- 4: Partition Q_t into group blocks B_t .
- 5: **for each** block $B \in B_t$ **do**
- 6: $S_B \leftarrow S$; $M_B \leftarrow \emptyset$.
- 7: **for each** group $Q \in B$ **do**
- 8: $M_Q \leftarrow \text{GREEDYGAINMATCHING}(Q, k, \tau_t, S_B)$.
- 9: $M_B \leftarrow M_B \cup M_Q$.
- 10: **end for**
- 11: $S \leftarrow \text{APPLYMERGEBATCH}(S, M_B)$.
- 12: $A \leftarrow \text{Active}(S)$.
- 13: **end for**
- 14: **end for**
- 15: $(P, C^+, C^-) \leftarrow \text{ENCODEFLATSUMMARY}(G, S)$.
- 16: **return** $\Sigma = (S, P, C^+, C^-)$.

Figure 1: BEAM repeatedly constructs group-local sparse candidate graphs under a frozen block state and commits the selected endpoint-disjoint merges at block granularity.

3.2.1 Adaptive Multi-Hash Divide. At the beginning of each iteration, BEAM regenerates divide groups from the current active supernodes. It computes locality-sensitive min-hash ranks over multiple dimensions and recursively refines oversized buckets until each group satisfies the configured size bound [6, 7]. If the available hash dimensions do not separate an oversized bucket, a deterministic fallback split completes the partition.

The divide stage bounds each local search domain and assigns every active supernode to exactly one group. A divide group determines candidate eligibility and group-local matching; a group block is only the frozen-state commit unit used to schedule several disjoint groups.

3.2.2 Sparse Profiles and EA-Proxy Retention. For each active supernode X , BEAM uses the sparse quotient profile

$$W_X[Z] = e(X, Z),$$

with zero coordinates omitted. Target buckets index the sources whose profiles contain the same nonzero coordinate. A pair (X, Y) becomes a proxy pair only when its profiles share at least one target,

$$B(X, Y) = \{Z : W_X[Z] > 0 \wedge W_Y[Z] > 0\}.$$

For a shared target Z , define the external pair-cost contribution

$$\Delta_Z(X, Y) = L(X, Z) + L(Y, Z) - L(X \cup Y, Z).$$

BEAM accumulates positive shared-target contributions together with a lightweight direct-edge signal:

$$\hat{g}(X, Y) = \sum_{Z \in B(X, Y)} \max\{0, \Delta_Z(X, Y)\} + e(X, Y). \quad (15)$$

Algorithm 2: Exact-Admissible Candidate Graph

Input: retained set C_Q ; profiles $\{W_X : X \in Q\}$; frozen state S_B ; threshold τ_t .
Output: weighted graph $H_{Q,t} = (Q, C_{Q,t}^{\text{adm}}, w)$.

- 1: $C_{Q,t}^{\text{adm}} \leftarrow \emptyset$; $w \leftarrow \emptyset$.
- 2: **for each** $\{X, Y\} \in C_Q$ **do**
- 3: Obtain the merged statistics in Equation (18).
- 4: $g \leftarrow \text{gain}_{S_B}(X, Y)$.
- 5: $\rho \leftarrow g / L_{\text{before}, S_B}(X, Y)$.
- 6: **if** $g > 0$ and $\rho \geq \tau_t$ **then**
- 7: $C_{Q,t}^{\text{adm}} \leftarrow C_{Q,t}^{\text{adm}} \cup \{\{X, Y\}\}$.
- 8: $w(X, Y) \leftarrow g$.
- 9: **end if**
- 10: **end for**
- 11: **return** $H_{Q,t} = (Q, C_{Q,t}^{\text{adm}}, w)$.

Figure 2: Every retained pair is evaluated against the same frozen state. The exact threshold determines admissibility, while exact gain defines the edge weight.

To avoid systematically favoring large supernodes, the accumulated score is normalized by profile-derived old-cost estimates,

$$\hat{\rho}(X, Y) = \frac{\hat{g}(X, Y)}{\tilde{C}_{\text{old}}(X) + \tilde{C}_{\text{old}}(Y) + \varepsilon}. \quad (16)$$

Let $\text{top}_k(X, Q)$ contain the at most k highest-ranked partners of X in Q . BEAM retains the symmetric union

$$C_Q = \left\{ \{X, Y\} \in \binom{Q}{2} : Y \in \text{top}_k(X, Q) \vee X \in \text{top}_k(Y, Q) \right\}. \quad (17)$$

Duplicate occurrences generated by different targets or endpoints are removed using a canonical pair order.

The proxy deliberately uses only a restricted and positively truncated view of the affected neighborhood. It therefore ranks search effort rather than estimating the exact retained-edge weight; every edge in C_Q proceeds to the exact decision stage.

3.3 Exact Encoding-Aware Decision

For each retained pair $\{X, Y\} \in C_Q$, BEAM obtains the statistics needed by the exact objective from the frozen block state S_B . If $M_{XY} = X \cup Y$, the merged statistics are

$$\begin{aligned} |M_{XY}| &= |X| + |Y|, \\ e(M_{XY}, M_{XY}) &= e(X, X) + e(Y, Y) + e(X, Y), \\ e(M_{XY}, Z) &= e(X, Z) + e(Y, Z). \end{aligned} \quad (18)$$

The last identity is evaluated only over the union of the sparse supports of W_X and W_Y .

Algorithm 2 applies the exact gain and relative-saving definitions from Section 2.4. By Equation (12), every retained weight is the exact partition-cost change of its hypothetical merge under S_B .

Algorithm 3: Greedy Gain Matching within One Group

Input: divide group Q ; candidate budget k ; threshold τ_t ; frozen state $\mathcal{S}_B$.

Output: conflict-free merge set $\mathcal{M}_Q$.

```

1: Construct the sparse profiles  $\{W_X : X \in Q\}$  under  $\mathcal{S}_B$ .
2:  $C_Q \leftarrow \text{PROFILECANDIDATESPARSIFICATION}(Q, \{W_X\}, k)$ .
3:  $H_{Q,t} \leftarrow \text{EXACTADMISSIBLEGRAPH}(C_Q, \{W_X\}, \mathcal{S}_B, \tau_t)$ .
4: Order  $C_{Q,t}^{\text{adm}}$  by non-increasing  $w$ .
5:  $\mathcal{M}_Q \leftarrow \emptyset$ ;  $\text{used} \leftarrow \emptyset$ .
6: for each  $\{X, Y\} \in C_{Q,t}^{\text{adm}}$  in this order do
7:   if  $X, Y \notin \text{used}$  then
8:      $\mathcal{M}_Q \leftarrow \mathcal{M}_Q \cup \{\{X, Y\}\}$ .
9:      $\text{used} \leftarrow \text{used} \cup \{X, Y\}$ .
10:  end if
11: end for
12: return  $\mathcal{M}_Q$ .
```

Figure 3: Group-local candidate proposal, exact decision, and conflict-free selection. Proxy scores are discarded before matching.

3.4 Greedy Exact-Admissible Matching

BEAM processes the edges of $H_{Q,t}$ in non-increasing exact gain, using a canonical endpoint order for ties, and accepts an edge whenever both endpoints are unused. Algorithm 3 combines proposal, exact decision, and conflict resolution for one divide group.

PROPOSITION 3.1 (RETAINED-GRAPH MATCHING GUARANTEE). *Fix $H_{Q,t} = (Q, C_{Q,t}^{\text{adm}}, w)$. Let $\mathcal{M}_Q^g$ be the matching returned by Algorithm 3, and let $\mathcal{M}_Q^*$ be a maximum-weight matching on the same graph. Then*

$$w(\mathcal{M}_Q^g) \geq \frac{1}{2}w(\mathcal{M}_Q^*).$$

PROOF. When greedy selects an edge $e = \{X, Y\}$, it can block at most two edges of $\mathcal{M}_Q^*$, one incident to each endpoint. Because edges are processed in non-increasing weight, every optimal edge first blocked by e has weight at most $w(e)$. Charging each optimal edge to the greedy edge that first blocks it assigns total charge at most $2w(e)$ to every selected edge. Summing the charges gives $w(\mathcal{M}_Q^*) \leq 2w(\mathcal{M}_Q^g)$. $\square$

The proposition measures only the loss due to greedy conflict resolution after $H_{Q,t}$ has been fixed. Candidate sparsification, divide-group eligibility, thresholding, and later partition evolution remain outside this local guarantee.

3.5 Complexity and Approximation Scope

For a group Q , let $B_Z = \{X \in Q : W_X[Z] > 0\}$ be the target bucket of coordinate Z , d_X the number of accumulated proxy partners of X , $c = |C_Q|$, and $a = |C_{Q,t}^{\text{adm}}|$. With deterministic full ranking, the group-local work is

$$O\left(\sum_Z |B_Z|^2 + \sum_{X \in Q} d_X \log d_X + \sum_{\{X,Y\} \in C_Q} |\text{supp}(W_X) \cup \text{supp}(W_Y)|\right) \quad (19)$$

where $c \leq |Q|k$. The four terms correspond to proxy-pair generation, per-source ranking, exact retained-edge evaluation, and greedy matching, respectively.

BEAM makes two explicit approximations: profile-based top- k retention sparsifies the current search space, and greedy selection approximates maximum-weight matching on the resulting exact-admissible graph. Retained-edge weights themselves are exact under the frozen state. These local properties do not compose into a global approximation ratio across groups, blocks, or iterations.

3.6 Blocked Merge Application

For a block B , the divide groups are disjoint, so

$$\mathcal{M}_B = \bigcup_{Q \in B} \mathcal{M}_Q$$

is endpoint-disjoint. All of its edges were proposed, evaluated, and selected against the same frozen state $\mathcal{S}_B$. BEAM commits the block by replacing every selected pair $\{X, Y\}$ with $M_{XY} = X \cup Y$ and updating its quotient statistics using Equation (18).

Endpoint-disjointness makes the partition mutation unambiguous; it does not make isolated edge gains additive. Once the block is committed, statistics derived from the old partition are stale, and subsequent blocks regenerate the required profiles, candidates, and weights from the updated state. After the final iteration, BEAM materializes (P, C^+, C^-) for the resulting partition.

4 SEMANTICS-PRESERVING EXECUTION

The preceding sections define BEAM’s candidate, decision, and blocked commit semantics. This section describes the execution substrate used to realize the same algorithm over an evolving partition. Its purpose is to provide current quotient statistics and evaluate retained pairs efficiently; it does not change candidate construction, the exact objective, positive-gain filtering, matching order, or block-commit semantics.

4.1 Persistent Quotient State

In persistent mode, BEAM maintains the current quotient graph instead of reconstructing supernode neighborhoods from the original CSR graph whenever a group is prepared. For each active supernode, the state stores its cardinality, internal edge count, sorted sparse quotient row, and active/version metadata. These quantities provide the current profile entries and the local statistics required by Section 3.3.

After the endpoint-disjoint merges of a block have been fixed, the quotient state is updated using the same count-addition rules described in Section 3.6. The rows of the two endpoints are merged, reciprocal entries of affected neighbors are updated, the removed endpoint is marked inactive, and the versions of modified rows are advanced. Read-only rows expose the maintained sparse rows without materializing group-local copies; version checks prevent a view from being reused after its row has changed.

The persistent representation is an execution optimization rather than a different summary model. At any frozen partition state, it exposes the same supernode sizes, internal counts, and quotient edge counts as a reconstruction from the original graph. It therefore preserves the profiles, retained candidates, exact gains, and selected merges of the corresponding rebuild-based execution. Avoiding reconstruction and row materialization removes deterministic work, although the resulting wall-clock benefit remains workload dependent and is evaluated separately.

4.2 Unified Exact Cost Interface and Batched Scoring

BEAM implements the objective of Section 2 through a common integer cost interface. The implementation exposes `BlockCost` for the local pair cost L , `ExactMergeGain` for Equation (11), and `ExactPartitionCost` for Equation (5). Reusing the same block-cost routine across local scoring and partition-level checks keeps the execution path aligned with the objective already used to define merge acceptance. The interface strengthens implementation consistency; it does not introduce an additional merge policy.

Once profile-based proposal has fixed a retained set C_Q , each $\{X, Y\} \in C_Q$ becomes an independent exact-scoring task against the same frozen state $\mathcal{S}_B$. The scorer reads the two current sparse quotient rows together with their sizes and internal counts, evaluates the exact gain defined earlier, and returns one result for the candidate. Parallelism is therefore across retained pairs, while the objective and the candidate graph remain unchanged.

For batched execution, one or more prepared groups are represented in a compact CSR-like layout containing row offsets, neighbor identifiers, edge counts, row sizes, internal counts, and candidate endpoint indices. Group offsets map returned scores back to their group-local candidate graphs. The CPU implementation distributes candidate tasks across worker threads. After scoring, the host applies the same positive-gain filtering and greedy matching from Section 3.4; batching changes only how the fixed set of exact evaluations is scheduled.

4.3 Optional CUDA Scoring Backend

The CUDA backend consumes the same packed scoring workload. Device workers evaluate retained pairs independently, with different workers possibly traversing different numbers of sparse row entries. The host transfers the packed quotient rows and candidate indices, launches the exact-gain kernel in candidate chunks when necessary, and copies the result array back. Candidate generation, filtering, matching, merge application, quotient updates, and final encoding remain outside the CUDA scorer.

CPU and CUDA are therefore interchangeable realizations of the same scoring interface rather than different BEAM algorithms. Device execution can benefit sufficiently large scoring batches, but transfer and launch costs, small candidate sets, and variation in sparse-row lengths can limit that

benefit. We consequently report CUDA results as a workload-dependent backend study rather than as the defining algorithmic contribution.

5 EXPERIMENTAL EVALUATION

5.1 Experimental Questions

Our evaluation follows the separation between proposal, exact decision, and semantics-preserving execution established in Sections 2–4. We ask six questions.

- **RQ1.** Does the implemented exact merge gain agree with an independently recomputed global objective, and do the resulting summaries remain lossless?
- **RQ2.** Which reconstruction and row-materialization work is eliminated by the persistent quotient state?
- **RQ3.** How strongly does proxy-based retention rely on exact validation, and what happens when the candidate path is replaced by more aggressively bounded alternatives?
- **RQ4.** How does BEAM compare with SWeG and SLUGGER in average compression ratio and average runtime?
- **RQ5.** Which stages dominate BEAM’s end-to-end runtime?
- **RQ6.** What practical role does the optional CUDA exact-scoring backend play?

The first three questions test the semantic and algorithmic claims of BEAM. RQ4 provides the primary end-to-end comparison, RQ5 explains where the time is spent, and RQ6 isolates an optional implementation backend rather than treating GPU execution as the central contribution.

5.2 Datasets, Baselines, and Reporting Protocol

We evaluate BEAM on eleven graph datasets spanning social, email, collaboration, product, road, and hyperlink networks. Table 2 summarizes their sizes. Most datasets are from SNAP [16], while the two largest web graphs are from the LAW/WebGraph ecosystem [5]. The end-to-end comparison uses the ten datasets for which SWeG, SLUGGER, and BEAM results are all available. Orkut is retained in the BEAM profiling studies but omitted from the three-way comparison because matching baseline results are unavailable.

SWeG is the principal same-representation baseline: it produces a flat correction-set summary and is the closest scalable algorithmic reference for BEAM [2, 12, 20–22]. We use the implementation released with the LDME project and its multithreaded reproduction. **SLUGGER** is a cross-representation reference based on a hierarchical summary. Its results are therefore interpreted as broader compression-runtime context rather than a same-semantics comparison. MAGS and MAGS-DM provide contemporary same-representation context, but the available records were collected under a different measurement environment and are not mixed into the controlled speedup table.

Table 2: Datasets used in the evaluation.

Dataset	Nodes	Edges	Type
Facebook(FA)	4,039	88,234	Social
Email-Enron(EM)	36,692	183,831	Email
Amazon(AM)	334,863	925,872	Network
DBLP(DB)	317,080	1,049,866	Collaboration
Youtube(YO)	1,134,890	2,987,624	Social
RoadNet(RN)	1,965,206	2,766,607	Network
Livejournal(LJ)	3,997,962	34,681,189	Social
Hollywood(HO)	1,107,243	56,375,711	Collaboration
Orkut(OR)	3,072,441	117,185,083	Network
UK-2002(U2)	18,520,343	261,787,258	Hyperlinks
UK-2005(U5)	39,459,923	783,027,125	Hyperlinks

The end-to-end experiments run on a dual-socket server with two Intel Xeon Silver 4316 CPUs at 2.30 GHz, providing 40 physical cores and 80 hardware threads, together with an NVIDIA Tesla P100 GPU. Unless otherwise stated, all methods run for 20 iterations. The reported blocked BEAM configuration uses $k = 16$, a group-block size of 64, and a candidate-batch budget of 65,536.

For each dataset-method pair in the main comparison, we report the average compression ratio and average wall-clock time over the available repeated runs. Randomized divide/grouping choices and ordinary system scheduling can produce small run-to-run changes in both the final partition and elapsed time; we treat these small differences as normal experimental perturbation and use the experiment-level averages rather than elevating individual runs into separate result categories. The available records do not retain a complete per-run variance series for every dataset, so we do not attach confidence intervals or statistical-significance claims. The deterministic correctness checks and bounded ablations reported below are stated separately under their own test scopes and are not numerically pooled with the end-to-end averages.

5.3 Metrics

The primary quality metric is the final correction-set cost normalized by the number of original edges:

$$\text{RelSize} = \frac{\text{Cost}(\mathcal{S}_{\text{final}})}{|E|},$$

where Cost is the exact pairwise sparse-or-dense objective from Equation (5). Lower relative size indicates better compression. Runtime is wall-clock execution time; the main comparison uses the mean end-to-end algorithm time of the available repeated runs.

For BEAM, we additionally report divide, merge, encode, prepare, and exact-gain scoring time. **Prepare** includes sparse-profile refresh, target-bucket construction, EA-proxy accumulation, per-source top- k retention, deduplication, and construction of the retained candidate representation. **Gain scoring** includes only exact evaluation of retained candidates. CUDA H2D, kernel, D2H, scoring-slice, and launch measurements are diagnostic counters and are not assumed

Table 3: Exact-objective and lossless-output validation. These are deterministic correctness checks, not performance measurements.

Check	Coverage	Result
Local gain equals global cost decrease	500 random graphs plus named families; every legal merge	Pass
64-bit objective and capacity semantics	Standalone oracle edge cases	Pass
Payload cost equals exact partition cost	7 named graphs $\times$ 2 CPU merge modes	Pass
Lossless reconstruction	Same 14 integration configurations	Pass
Persistent update equals full rebuild	300 random graphs and multiple merge orders	Pass
Persistent and legacy output equality	7 named graphs	Byte-identical

to form a strictly additive decomposition because streamed execution can overlap components.

5.4 RQ1: Exact-Objective and Output Fidelity

Table 3 connects the local gain implementation to the global objective and the materialized summary. For every legal merge in 500 fixed-seed random small graphs and a set of named graph families, the integer local gain equaled the independently recomputed difference between the global partition cost before and after the merge. Integration tests on seven named graphs and two CPU merge modes further matched the payload cost, the exact partition cost, and the reported MAGS-compatible cost, and the emitted $G/P/C^+/C^-$ payload reconstructed the input graph losslessly. Finally, 300 random-graph update sequences matched a full quotient-state rebuild, and the frozen seven-graph gate produced byte-identical persistent and legacy payloads.

These tests support objective fidelity for the CPU MAGS-compatible path and the lossless summary representation. They do not constitute a formal proof for all inputs, nor do they imply that every optional backend has been validated under every configuration.

5.5 RQ2: Persistent-State Work Elimination

The persistent quotient state is intended to preserve algorithmic semantics while avoiding repeated reconstruction from the original CSR. Table 4 reports deterministic work counters. In the legacy prepare path, repeated reconstruction corresponds to 16.461–19.653 full $2|E|$ -equivalent scans over the seven profiled datasets. In the persistent path, the post-initialization original-CSR scan count is zero on all seven. The CPU row-view milestone also reports zero prepared-row entries and zero bytes copied. Thus the implementation removes the measured reconstruction and row-materialization work rather than merely hiding it behind a faster kernel.

Table 4: Original-CSR reconstruction work in the legacy and persistent prepare paths. The two columns are deterministic work profiles, not paired runtime measurements.

Dataset	Legacy $2 E $ -equiv. scans	Persistent post-init scans
FA	19.653	0
EM	18.437	0
AM	17.815	0
DB	17.992	0
YO	17.250	0
RN	16.461	0
LJ	19.131	0

Table 5: Default Facebook candidate funnel after proxy/top- k retention.

Measurement	Count
Retained pairs sent to exact scoring	289,013
Retained pairs with positive exact gain	99,331
Retained pairs removed by exact validation	189,682
Positive fraction	34.4%

On the four datasets with adjacent paired measurements, the persistent-to-legacy runtime ratios were 0.993, 0.981, 1.031, and 1.025 for FA, EM, AM, and DB, respectively. These near-parity measurements show that the first persistent implementation did not introduce a large regression, but we do not infer a proportional runtime speedup from the eliminated-work counters.

5.6 RQ3: Candidate Screening and Merge-Trajectory Boundaries

BEAM uses the EA proxy to allocate a bounded exact-evaluation budget; it does not use the proxy as the merge objective. The frozen Facebook funnel in Table 5 makes this distinction concrete. The default path retained and exact-scored 289,013 pairs, but only 99,331 had positive exact gain. Consequently, 189,682 retained pairs were removed by exact validation. The positive fraction was 34.4%. This aggregate funnel does not measure proxy rank correlation, exact-best recall, or regret, because per-pair proxy scores and ranks were not retained.

Table 6 then evaluates bounded alternative candidate paths on Facebook. The quotient-neighbor path with budget 2 reduced exact calls from 274,635 to 118,226, yet worsened the final ratio from 0.483725 to 0.704672 and increased runtime. The residual-signature path also reduced exact calls, to 158,451, but ended at a ratio of 0.510619 and a higher runtime. Thus, reducing exact-call count alone is not a sufficient optimization target: the changed proposal graph alters the long-horizon merge trajectory and can lose more in final partition quality than it saves in local evaluation work. These are bounded single-dataset negative ablations; they do not establish that all alternative candidate indexes fail on every graph.

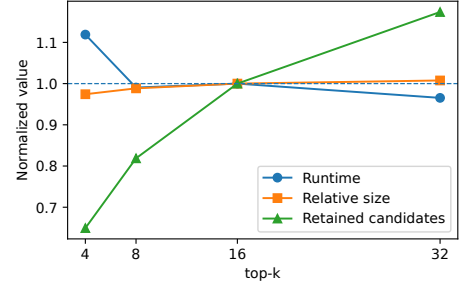


Figure 4: RQ3: Sensitivity to top- k on five datasets. Values are geometric means normalized to $k = 16$. Larger budgets retain more candidates, while compression improves only slightly beyond the default.

Parameter sensitivity. Figure 4 evaluates the primary proposal-budget parameter on FA, EM, AM, DB, and YO. Values are geometric means normalized to $k = 16$. Smaller k retains fewer candidates but degrades compression, whereas increasing k from 16 to 32 yields only a small relative-size improvement while substantially increasing retained work. We therefore use $k = 16$ as the fixed default. Group-block sizes in $\{32, 64, 128, 256\}$ and candidate-batch budgets from 4,096 to 65,536 produced essentially unchanged compression; their runtime differences mainly reflect batching and launch overhead.

5.7 RQ4: End-to-End Compression–Runtime Tradeoff

Table 7, Figure 5, and Figure 6 report the average compression ratio and average runtime on the ten common datasets. SWeG is the primary comparison because it uses the same flat correction-set representation. BEAM is faster than SWeG on all ten datasets, with a geometric-mean speedup of $2.69\times$, and produces a smaller average relative size on all ten. This result supports the main same-model claim: sparse candidate proposal, exact retained-edge evaluation, and conflict-free blocked selection improve the observed speed–quality tradeoff without changing the summary representation.

Against SLUGGER, BEAM is faster on nine of ten datasets, with a geometric-mean speedup of $2.29\times$; RoadNet is the exception. BEAM produces a smaller average relative size on eight datasets, while SLUGGER is smaller on FA and HO. Because SLUGGER uses a hierarchical representation, this comparison provides cross-model context rather than evidence that one representation strictly dominates the other.

5.8 RQ5: Runtime Concentration in the Merge Stage

Figure 7 and Table 8 show that merge-related work accounts for 50.0%–97.9% of BEAM’s measured runtime. The combined prepare-and-gain-scoring share is 21.3%–79.4%, whereas exact gain scoring alone is 2.0%–19.0%. Candidate preparation is therefore often at least as important as the exact

Table 6: Facebook candidate-path ablation. Fewer exact evaluations do not necessarily preserve final partition quality or reduce runtime.

Path	Budget	Exact calls	Merges	Exact cost	Rel. size	Algorithm time (ms)
Legacy CandidateIndex	16	274,635	2,346	42,681	0.483725	328.689
Quotient-neighbor	2	118,226	1,484	—	0.704672	494.555
Quotient-neighbor	4	194,093	2,049	—	0.582508	425.678
Quotient-neighbor	8	351,503	2,256	—	0.551930	456.731
Residual-signature	4	158,451	2,317	45,054	0.510619	549.620

Table 7: RQ4: Mean end-to-end compression ratio and runtime on the ten common datasets. Speedups are computed from the reported mean runtimes. Lower is better.

Dataset	SWeG Rel. Size	SWeG Time (s)	SLUGGER Rel. Size	SLUGGER Time (s)	BEAM Rel. Size	BEAM Time (s)	Speedup vs SWeG	Speedup vs SLUGGER
FA	0.657	3.27	0.442	2.31	0.497	1.87	1.75×	1.24×
EM	0.891	5.03	0.763	7.63	0.711	3.99	1.26×	1.91×
AM	0.879	10.55	0.802	11.11	0.619	10.35	1.02×	1.07×
DB	0.698	15.34	0.690	13.82	0.571	10.11	1.52×	1.37×
YO	0.815	1127.86	0.935	103.32	0.735	68.90	16.37×	1.50×
RN	0.999	87.65	1.000	46.89	0.716	62.36	1.41×	0.75×
LJ	0.865	435.19	0.764	1061.45	0.756	299.03	1.46×	3.55×
HO	0.598	987.03	0.445	3525.73	0.549	263.78	3.74×	13.37×
U2	0.173	9442.87	0.144	8071.43	0.126	2024.26	4.66×	3.99×
U5	0.130	45681.93	0.110	24170.22	0.101	4585.81	9.96×	5.27×
Geo. mean speedup	—	—	—	—	—	—	2.69×	2.29×

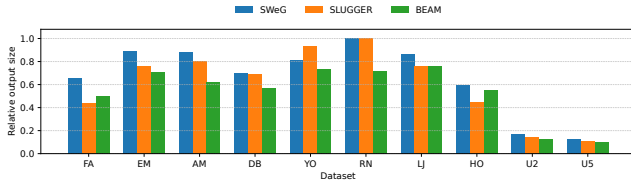


Figure 5: RQ4: Average relative output size on the ten common datasets; lower is better. BEAM improves over the same-model SWeG baseline on all ten datasets, while SLUGGER shows a mixed cross-model compression tradeoff.

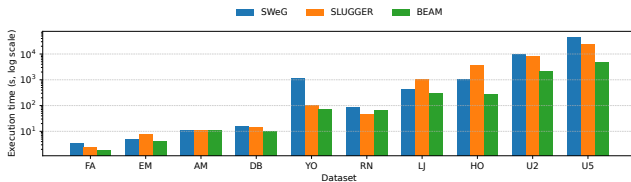


Figure 6: RQ4: Average execution time on the ten common datasets under 20 iterations. The y-axis is logarithmic; lower is better.

scorer. This result motivates the algorithm–system separation in Section 4: batching the exact objective helps, but end-to-end scalability also depends on persistent state, sparse proposal construction, matching, and merge application.

The runtime breakdown and the deterministic work counters in Table 4 answer different questions. The former shows

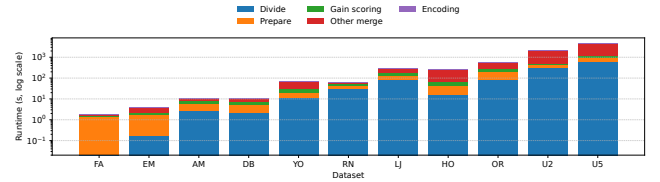


Figure 7: RQ5: BEAM runtime breakdown into divide, prepare, exact gain scoring, other merge work, and final encoding.

where wall time is spent in the reported end-to-end configuration; the latter shows which reconstruction work the persistent implementation removes. We do not infer a persistent-state speedup by subtracting or combining these two measurement scopes.

5.9 RQ6: Optional CUDA Exact-Scoring Backend

The recorded CPU/CUDA study uses the same high-level blocked pipeline and changes the retained exact-gain scoring backend. Small compression differences between repeated CPU and CUDA records are treated as ordinary run-to-run perturbation; consequently, this subsection reports average runtime behavior and does not make a backend-level compression claim. CUDA is evaluated as an optional scoring implementation, not as the source of BEAM’s candidate-search or selection semantics.

Table 9 shows that the CUDA-backed implementation is faster on nine of eleven datasets and has a geometric-mean end-to-end speedup of 1.19×. It is slower on FA and EM,

Table 8: RQ5: BEAM runtime breakdown across datasets.

Dataset	Total (s)	Divide (s)	Merge (s)	Encode (s)	Prepare (s)	Gain Scoring (s)	Merge Share	Prep.+Gain Share	Gain Share	Candidates
FA	1.87	0.02	1.83	0.02	0.36	0.04	97.9%	21.3%	2.0%	11,713
EM	3.99	0.17	3.79	0.04	1.82	0.32	94.8%	53.7%	8.0%	359,890
AM	10.35	2.65	7.48	0.22	2.32	1.82	72.3%	40.1%	17.6%	299,875
DB	10.11	2.18	7.68	0.24	2.47	1.92	76.0%	43.5%	19.0%	380,762
YO	68.90	11.33	56.84	0.71	38.33	10.56	82.5%	70.9%	15.3%	13,548,785
RN	62.36	30.24	31.20	0.88	8.55	11.53	50.0%	32.2%	18.5%	862,286
LJ	299.03	81.19	208.04	9.73	112.69	44.28	69.6%	52.5%	14.8%	4,175,972
HO	263.78	15.69	236.21	11.87	183.70	25.65	89.5%	79.4%	9.7%	7,659,141
OR	581.63	82.22	467.60	31.77	273.60	69.38	80.4%	59.0%	11.9%	1,801,651
U2	2024.26	303.31	1679.29	41.29	1465.70	99.32	83.0%	77.3%	4.9%	310,296,688
U5	4585.81	651.09	3789.20	144.87	3280.82	231.95	82.6%	76.6%	5.1%	823,853,836

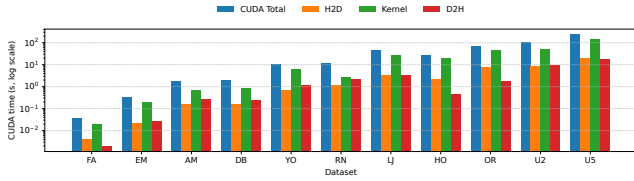


Figure 8: RQ6: CUDA scoring-path wall time and component counters. H2D, kernel, and D2H counters need not sum to CUDA Total under overlapped execution.

where transfer and launch overhead outweigh the small scoring workload. The largest gains appear on AM, DB, RN, and LJ. Since exact scoring occupies only 2.0%–19.0% of total runtime, the end-to-end improvement is necessarily bounded by the CPU-side proposal, prepare, matching, update, and encoding stages.

Figure 8 and Table 10 further separate CUDA wall time into diagnostic counters. On U5, kernel execution takes 136.373 s and H2D transfer takes 19.298 s. The 1,618,975 scoring slices and 1,607,709 launches reflect repeated blocked and sub-batched invocations rather than one monolithic kernel. Because transfers and kernels may overlap, the component counters need not sum to CUDA Total.

5.10 Evidence Scope and Remaining Measurements

The current evidence supports four bounded conclusions: the CPU exact objective matches independently recomputed global deltas on the tested graphs; persistent state eliminates the measured post-initialization reconstruction and row-copy work; exact validation removes many nonpositive proxy-retained candidates; and aggressive exact-call reduction can damage the final merge trajectory. The end-to-end table reports the available average compression and runtime results, with small run-to-run differences treated as normal perturbation.

Three claims remain outside the evidence reported here. First, the default EA proxy has no retained per-pair score/rank

trace, so we do not report recall, rank correlation, or regret for that proxy. Second, the available MAGS/MAGS-DM records are not used for strict runtime speedups because their measurement environment is not aligned with the controlled table. Third, the CUDA study is an implementation profile; the algorithmic claims do not depend on CUDA.

6 RELATED WORK

Graph compression and graph summarization. Graph compression reduces the physical footprint of a graph by exploiting regularities in adjacency lists and vertex orderings. Representative techniques include reference and interval encoding, locality-aware relabeling, layered encodings, recursive bisection, and computation directly over compressed layouts [1, 3–5, 8, 10, 17, 24]. Their output remains an encoding of the original adjacency relation. Structural graph summarization instead replaces vertices by aggregate objects and records enough residual information to reconstruct or approximate the input [18, 20]. BEAM belongs to the latter line: its target is a lossless flat summary with explicit supernode, superedge, and correction-set semantics, rather than a new low-level adjacency code.

Flat correction-set summarization. Navlakha et al. formulated the summary-plus-corrections model and proposed global greedy and randomized construction procedures [20]. Later work shifted attention from summary definition to scalable construction. SWeG introduced a divide-and-merge paradigm that restricts pairwise comparisons and supports shared-memory and distributed execution at web scale [22]. LDME further reduces merge-search overhead using weighted locality-sensitive hashing and a scalable divide-merge-encode pipeline [28]. MoSSo addresses a different setting, maintaining a flat lossless summary under edge insertions and deletions by incrementally relocating vertices among supernodes [13]. Most recently, MAGS and MAGS-DM revisit the compactness–efficiency tradeoff: MAGS accelerates the classical greedy paradigm, whereas MAGS-DM develops a faster divide-and-merge alternative under the same summary-and-corrections representation [9].

BEAM is positioned within this flat, lossless family, but differs in where approximation is permitted. Its lightweight profiles and top- k rule determine only which pairs enter the

Table 9: RQ6: Mean end-to-end time with CPU and CUDA exact-scoring backends in the blocked BEAM pipeline. CUDA scoring share is `cuda_total_ms/runtime_end_to_end_ms`.

Dataset	CPU time (s)	CUDA time (s)	E2E speedup	CUDA scoring share
FA	0.58	1.87	$0.31\times$	2.0%
EM	3.42	3.99	$0.86\times$	8.0%
AM	20.38	10.35	$1.97\times$	17.6%
DB	19.09	10.11	$1.89\times$	19.0%
YO	90.44	68.90	$1.31\times$	15.3%
RN	163.27	62.36	$2.62\times$	18.5%
LJ	397.39	299.03	$1.33\times$	14.8%
HO	276.52	263.78	$1.05\times$	9.7%
OR	663.50	581.63	$1.14\times$	11.9%
U2	2254.13	2024.26	$1.11\times$	4.9%
U5	4993.09	4585.81	$1.09\times$	5.1%

Table 10: RQ6: CUDA scoring-path measurements. CUDA Total is the high-level scoring wall time; component counters may overlap.

Dataset	Candidates	Block Size	CUDA Total (s)	H2D (s)	Kernel (s)	D2H (s)	Scoring Slices	Kernel Launches	Total Time (s)	CUDA Share
FA	11,713	64	0.037	0.004	0.020	0.002	168	168	1.87	2.0%
EM	359,890	64	0.319	0.022	0.188	0.027	2,774	2,774	3.99	8.0%
AM	299,875	64	1.822	0.162	0.670	0.267	28,160	28,148	10.35	17.6%
DB	380,762	64	1.922	0.157	0.842	0.248	26,031	26,025	10.11	19.0%
YO	13,548,785	64	10.550	0.652	5.814	1.106	116,126	115,821	68.90	15.3%
RN	862,286	64	11.512	1.139	2.606	2.142	229,609	229,476	62.36	18.5%
LJ	4,175,972	64	44.245	3.231	27.546	3.302	339,114	336,759	299.03	14.8%
HO	7,659,141	64	25.648	2.256	18.694	0.449	40,442	40,442	263.78	9.7%
OR	1,801,651	64	69.360	7.302	46.504	1.658	138,616	136,932	581.63	11.9%
U2	310,296,688	64	99.230	8.474	52.065	9.485	928,803	922,330	2024.26	4.9%
U5	823,853,836	64	231.791	19.298	136.373	17.071	1,618,975	1,607,709	4585.81	5.1%

retained candidate graph; they do not decide whether a retained merge is beneficial. Every retained edge is reweighted by the exact correction-set gain, nonpositive edges are discarded, and conflicts are resolved by gain-ordered matching before a frozen-state block commit. Thus, BEAM does not enlarge the representation class or replace the correction-set objective. It contributes an explicit separation between sparse proposal, exact retained-pair decision, conflict-free selection, and semantics-preserving state evolution.

Alternative objectives and representation models. Not all methods called graph summarization optimize the same object. Reconstruction-error methods seek a summary of prescribed size or trade compactness against approximation quality [2, 14]; SSumM, for example, jointly merges vertices and sparsifies the summary under an MDL-guided lossy objective [14]. Other work changes the representation itself. Hierarchical summarization allows nested supernodes and can

capture structures that a flat partition cannot express; SLUGGER instantiates this model with positive, negative, and hierarchy edges and uses sampling, approximation, and memoization for scalable construction [15, 26]. These approaches are important context, but their objective or model differs from BEAM’s flat lossless correction-set formulation. We therefore treat SWeG, LDME, and MAGS/MAGS-DM as same-representation neighbors, while SLUGGER provides a cross-model comparison.

Parallel graph processing. Parallel graph frameworks such as Ligra, Gunrock, and GraphBLAST accelerate traversal, frontier operations, and sparse linear algebra over a largely fixed graph representation [19, 23, 25, 27]. BEAM’s execution problem is different: each accepted merge changes the quotient state and thereby changes the vertices, edges, and weights of subsequent candidate graphs. Its persistent quotient representation and batched scoring interface are therefore specialized to repeated optimization over an evolving

summary state. CPU and CUDA scorers implement the same exact retained-edge objective; they are execution backends, not alternative summarization algorithms.

7 CONCLUSION

We presented BEAM, an objective-faithful sparse merging method for lossless flat correction-set graph summarization. BEAM approximates the evolving candidate search space rather than the merge objective: profile-based grouping and top- k retention bound exact work, while every retained candidate is evaluated by the exact local correction-set gain. Positive-gain, endpoint-disjoint matching and frozen-state block commits make the selection and update semantics explicit as the summary evolves.

The same separation carries into the system design. A unified cost oracle aligns local decisions with global partition cost and final encoding, a persistent quotient state avoids repeated reconstruction, and a backend-neutral batch interface exposes exact scoring to multicore CPU and optional CUDA execution without changing the algorithm. The evaluation verifies local-to-global objective agreement and lossless reconstruction on the tested instances, shows the reconstruction work removed by persistent state, and demonstrates that proxy retention still requires exact validation. The negative candidate-path ablations further show that reducing exact-call count in isolation can degrade the long-horizon merge trajectory.

Across the ten common datasets, BEAM improves average relative size over SWeG and reduces average runtime on every dataset, with a geometric-mean speedup of $2.69\times$. Against hierarchical SLUGGER, BEAM is faster on nine datasets with a $2.29\times$ geometric-mean speedup, while compression remains mixed across the two representation classes. Future work will study stronger candidate-search mechanisms and their effects on long-horizon merge trajectories.

ACKNOWLEDGMENTS

This work was supported in part by the National Key R&D Program of China under Grant No. 2023YFB3002702 and in part by the Creative Research Groups Program of the National Natural Science Foundation of China under Grant No. 62321003. Corresponding author: Guoqing Xiao.

REFERENCES

- [1] Alberto Apostolico and Guido Drovandi. 2009. Graph Compression by BFS. *Algorithms* 2, 3 (2009), 1031–1044.
- [2] Maham Anwar Beg, Muhammad Ahmad, Arif Zaman, and Imdadullah Khan. 2018. Scalable Approximation Algorithm for Graph Summarization. In *Advances in Knowledge Discovery and Data Mining - 22nd Pacific-Asia Conference, PAKDD 2018, Melbourne, VIC, Australia, June 3-6, 2018, Proceedings, Part III (Lecture Notes in Computer Science)*, Dinh Q. Phung, Vincent S. Tseng, Geoffrey I. Webb, Bao Ho, Mohadeseh Ganji, and Lida Rashidi (Eds.). Springer, 502–514.
- [3] Maciej Besta and Torsten Hoefer. 2018. Survey and Taxonomy of Lossless Graph Compression and Space-Efficient Graph Representations. *CoRR* abs/1806.01799 (2018).
- [4] Paolo Boldi, Marco Rosa, Massimo Santini, and Sebastiano Vigna. 2011. Layered label propagation: a multiresolution coordinate-free ordering for compressing social networks. In *Proceedings of the 20th International Conference on World Wide Web (Hyderabad, India) (WWW '11)*. Association for Computing Machinery, New York, NY, USA, 587–596.
- [5] Paolo Boldi and Sebastiano Vigna. 2004. The WebGraph Framework I: Compression Techniques. In *Proceedings of the 13th International Conference on World Wide Web, WWW 2004, New York, NY, USA, May 17-22, 2004*, Stuart I. Feldman, Mike Uretsky, Marc Najork, and Craig E. Wills (Eds.). ACM, 595–602.
- [6] Andrei Z. Broder. 1997. On the resemblance and containment of documents. In *Compression and Complexity of SEQUENCES 1997, Positano, Amalfitan Coast, Salerno, Italy, June 11-13, 1997, Proceedings*, Bruno Carpentieri, Alfredo De Santis, Ugo Vaccaro, and James A. Storer (Eds.). IEEE, 21–29.
- [7] Andrei Z. Broder, Moses Charikar, Alan M. Frieze, and Michael Mitzenmacher. 2000. Min-Wise Independent Permutations. *J. Comput. Syst. Sci.* 60, 3 (2000), 630–659.
- [8] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, Michael Mitzenmacher, Alessandro Panconesi, and Prabhakar Raghavan. 2009. On Compressing Social Networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2009, Paris, France, June 28 - July 1, 2009*. ACM, 219–228.
- [9] Deming Chu, Fan Zhang, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2024. Graph Summarization: Compactness Meets Efficiency. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.
- [10] Laxman Dhulipala, Igor Kabiljo, Brian Karrer, Giuseppe Ottaviano, Sergey Pupyrev, and Alon Shalita. 2016. Compressing Graphs and Indexes with Recursive Graph Bisection. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2016, San Francisco, CA, USA, August 13-17, 2016*. ACM, 1535–1544.
- [11] Harold N. Gabow. 1983. An Efficient Reduction Technique for Degree-Constrained Subgraph and Bidirected Network Flow Problems. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, STOC 1983, Boston, MA, USA, April 25-27, 1983*. ACM, 448–456.
- [12] Kifayat-Ullah Khan, Waqas Nawaz, and Young-Koo Lee. 2015. Set-based Approximate Approach for Lossless Graph Summarization. *Computing* 97, 12 (2015), 1185–1207.
- [13] Jihoon Ko, Yunbum Kook, and Kijung Shin. 2020. Incremental Lossless Graph Summarization. In *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*, Rajesh Gupta, Yan Liu, Jiliang Tang, and B. Aditya Prakash (Eds.). ACM, 317–327.
- [14] Kyuhan Lee, Hyeonsoo Jo, Jihoon Ko, Sungsu Lim, and Kijung Shin. 2020. SSumM: Sparse Summarization of Massive Graphs. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Virtual Event, CA, USA) (KDD '20)*. Association for Computing Machinery, New York, NY, USA, 144–154.
- [15] Kyuhan Lee, Jihoon Ko, and Kijung Shin. 2022. SLUGGER: Lossless Hierarchical Summarization of Massive Graphs. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 472–484.
- [16] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>. Accessed for benchmark graph datasets.
- [17] Yongsub Lim, U Kang, and Christos Faloutsos. 2014. Slashburn: Graph compression and mining beyond caveman communities. *IEEE Transactions on Knowledge and Data Engineering* 26, 12 (2014), 3077–3089.
- [18] Yike Liu, Tara Safavi, Abhilash Dighe, and Danai Koutra. 2018. Graph Summarization Methods and Applications: A Survey. *ACM Comput. Surv.* 51, 3 (2018), 62:1–62:34.
- [19] Duane Merrill, Michael Garland, and Andrew S. Grimshaw. 2012. Scalable GPU graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2012, New Orleans, LA, USA, February 25-29, 2012*, J. Ramanujam and P. Sadayappan (Eds.). ACM, 117–128.
- [20] Saket Navlakha, Rajeev Rastogi, and Nisheeth Shrivastava. 2008. Graph summarization with bounded error. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, Jason Tsong-Li Wang (Ed.). ACM, 419–432.

- [21] Hojin Seo, Kisung Park, Yongkoo Han, Hyunwook Kim, Muhammad Umair, Kifayat Ullah Khan, and Young Koo Lee. 2020. An Effective Graph Summarization and Compression Technique for a Large-scaled Graph. *J. Supercomput.* 76, 10 (2020), 7906–7920.
- [22] Kijung Shin, Amol Ghoting, Myunghwan Kim, and Hema Raghavan. 2019. SWeG: Lossless and Lossy Summarization of Web-Scale Graphs. In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, Ling Liu, Ryen W. White, Amin Mantrach, Fabrizio Silvestri, Julian J. McAuley, Ricardo Baeza-Yates, and Leila Zia (Eds.). ACM, 1679–1690.
- [23] Julian Shun and Guy E. Blelloch. 2013. Ligra: a lightweight graph processing framework for shared memory. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP '13, Shenzhen, China, February 23-27, 2013*, Alex Nicolau, Xiaowei Shen, Saman P. Amarasinghe, and Richard W. Vuduc (Eds.). ACM, 135–146.
- [24] Julian Shun, Laxman Dhulipala, and Guy E. Blelloch. 2015. Smaller and Faster: Parallel Processing of Compressed Graphs with Ligra+. In *2015 Data Compression Conference, DCC 2015, Snowbird, UT, USA, April 7-9, 2015*. IEEE, 403–412.
- [25] Yangzihao Wang, Andrew A. Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. 2016. Gunrock: a high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2016, Barcelona, Spain, March 12-16, 2016*, Rafael Asenjo and Tim Harris (Eds.). ACM, 11:1–11:12.
- [26] Rui Yan, Zi Yuan, Xiaojun Wan, Yan Zhang, and Xiaoming Li. 2012. Hierarchical Graph Summarization: Leveraging Hybrid Information through Visible and Invisible Linkage. In *Advances in Knowledge Discovery and Data Mining*, Pang-Ning Tan, Sanjay Chawla, Chin Kuan Ho, and James Bailey (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 97–108.
- [27] Carl Yang, Aydin Buluç, and John D. Owens. 2022. Graph-BLAST: A High-Performance Linear Algebra-based Graph Framework on the GPU. *ACM Trans. Math. Softw.* 48, 1 (2022), 1:1–1:51.
- [28] Quinton Yong, Mahdi Hajiabadi, Venkatesh Srinivasan, and Alex Thomo. 2021. Efficient Graph Summarization using Weighted LSH at Billion-Scale (*SIGMOD '21*). Association for Computing Machinery, New York, NY, USA, 2357–2365.